AI - Assisted Coding-Assignment -21.2

HTNO:2303A52228
Batch:37

Lab 21 - Collaborative AI Coding: Pair Programming and Version Control Integration.

Task Description-1
AI-Assisted Pair Programming.
Expected Output:
Compare the original and improved versions to understand the role of AI in collaboration.
Prompt:
#Generate a simple Python To-Do List Manager that allows users to:
#1. Add tasks
#2. View tasks
#3. Delete tasks
#Use a basic list and menu-driven approach.
CODE AND OUTPUT :

```python
class ToDoListManager:
    def __init__(self):
        self.tasks = []
    def add_task(self, task):
        self.tasks.append(task)
        print(f'Task "{task}" added to the list.')
    def view_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
        else:
            print("To-Do List:")
            for idx, task in enumerate(self.tasks, start=1):
                print(f"{idx}. {task}")
    def delete_task(self, task_number):
        if 0 < task_number <= len(self.tasks):
            removed_task = self.tasks.pop(task_number - 1)
            print(f'Task "{removed_task}" deleted from the list.')
        else:
            print("Invalid task number.")
def main():
    manager = ToDoListManager()
    while True:
        print("\nTo-Do List Manager")
        print("1. Add Task")
        print("2. View Tasks")
```

```
To-Do List Manager
1. Add Task
2. View Tasks
3. Delete Task
4. Exit
Enter your choice: 1
Enter the task to add: Writing
Task "Writing" added to the list.

To-Do List Manager
1. Add Task
2. View Tasks
3. Delete Task
4. Exit
Enter your choice: 1
Enter the task to add: Eating
Task "Eating" added to the list.

To-Do List Manager
1. Add Task
2. View Tasks
3. Delete Task
4. Exit
Enter your choice: 2
To-Do List:
1. Writing
2. Eating
```

Explanation:
Initially, the program was basic and could easily crash if the user entered invalid input. Using AI as a pair programmer, the code was improved by adding error handling, input validation, and better structure using functions. This made the program more reliable, user-friendly, and closer to real-world application standards.

Task Description-2
Version Control Integration.
Expected Output:
Reflect on how version control supports collaborative development with AI assistance.
Prompt:

Generate a Python script that connects to an external API using a hardcoded API key.
1. Identify the security risks of hardcoded API keys
2. Refactor the code to use environment variables instead of hardcoding
3. Add proper error handling
4. Show how to set environment variables on Windows/Linux
5. Provide sample input and output.

CODE AND OUTPUT :

```python
tasks = []
# Add task
def add_task(task):
    tasks.append(task)
    return "Task added"
# View tasks
def view_tasks():
    return tasks
# Run program
if __name__ == "__main__":
    print(add_task("Complete Lab"))
    print(add_task("Practice Git"))
    print("Tasks:", view_tasks())
# AI improvement: count tasks
def count_tasks():
    return len(tasks)
if __name__ == "__main__":
    print(add_task("Complete Lab"))
    print(add_task("Practice Git"))
    print("Tasks:", view_tasks())
    print("Total Tasks:", count_tasks())
```

```
Task added
Task added
Tasks: ['Complete Lab', 'Practice Git']
Task added
Task added
Tasks: ['Complete Lab', 'Practice Git', 'Complete Lab', 'Practice Git']
Total Tasks: 4
```

Explanation:

In this task, version control is used to manage changes in the project step by step. First, the basic Python program is created and saved. Then, using Git, a new branch is created to add improvements suggested by AI, such as counting the number of tasks.
Instead of directly modifying the main code, changes are made in a separate branch. This ensures that the original code remains safe. After testing the new feature, the changes are committed with a meaningful message and merged back into the main branch.

Task Description-3
AI for Commit Message Generation.
Expected Output:
Discuss how AI can improve documentation and traceability in Git history.
Prompt:
Generate concise and meaningful Git commit messages for the following changes:
1. Created a basic To-Do List program
2. Added exception handling and input validation
3. Refactored code into functions
4. Fixed invalid input and empty task issues.
CODE AND OUTPUT:

```python
def generate_commit_message(change):
    change = change.lower()
    if "add" in change or "create" in change:
        return "Add new feature: " + change
    elif "fix" in change or "bug" in change:
        return "Fix issue: " + change
    elif "improve" in change or "enhance" in change:
        return "Enhance functionality: " + change
    elif "refactor" in change:
        return "Refactor code: " + change
    else:
        return "Update code: " + change
changes = [
    "Created basic To-Do list program",
    "Added input validation and exception handling",
    "Refactored code into functions",
    "Fixed invalid input bug"
]
print("AI Generated Commit Messages:\n")
for i in range(len(changes)):
    msg = generate_commit_message(changes[i])
    print(str(i+1) + ".", msg)
```

```
AI Generated Commit Messages:


1. Add new feature: created basic to-do list program
2. Add new feature: added input validation and exception handling
3. Refactor code: refactored code into functions
4. Fix issue: fixed invalid input bug
```

Explanation:
This program acts like a simple AI that generates commit messages based on the type of code change. It checks keywords like "add," "fix," or "refactor" and creates a meaningful commit message automatically. This helps developers write clear and consistent commit messages, improving project tracking and collaboration.

Task Description-4
Pair Programming Simulation.
Expected Output:

Both students collaborate using Git branching and merging to integrate their work.
Prompt:
Simulate pair programming between two students:Student A writes a basic Python program (e.g., To-Do List or Calculator).Student B improves it using AI by adding features like validation, exception handling, and better structure,Show both versions of the code.Explain improvements clearly.
CODE AND OUTPUT :

```python
tasks = []
while True:
    print("\n1. Add Task")
    print("2. View Tasks")
    print("3. Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        task = input("Enter task: ")
        tasks.append(task)
    elif choice == 2:
        for i in range(len(tasks)):
            print(i + 1, tasks[i])
    elif choice == 3:
        break
```

```
1. Add Task
2. View Tasks
3. Exit
Enter choice: 1
Enter task: Eating

1. Add Task
2. View Tasks
3. Exit
Enter choice: 2
1 Eating
```

```python
tasks = []
def add_task():
    task = input("Enter task: ")
    if task.strip() == "":
        print("Task cannot be empty!")
    else:
        tasks.append(task)
        print("Task added successfully!")
def view_tasks():
    if len(tasks) == 0:
        print("No tasks available.")
    else:
        print("\nYour Tasks:")
        for i in range(len(tasks)):
            print(i + 1, tasks[i])
while True:
    try:
        print("\n1. Add Task")
        print("2. View Tasks")
        print("3. Exit")
        choice = int(input("Enter choice: "))
        if choice == 1:
            add_task()
        elif choice == 2:
            view_tasks()
        elif choice == 3:
```

```
1. Add Task
2. View Tasks
3. Exit
Enter choice: 1
Enter task: Eating
Task added successfully!

1. Add Task
2. View Tasks
3. Exit
Enter choice: 2

Your Tasks:
1 Eating
```

Explanation:
The initial program created by Student A is a simple To-Do list where users can add and view tasks, but it lacks proper error handling and structure. If the user enters invalid input, the program may crash. Student B improved the program using AI by organizing the code into functions, which makes it cleaner and easier to understand. They also added validation to prevent empty tasks and included exception handling to manage invalid inputs without crashing the program. Overall, the improved version is more user-friendly, reliable, and closer to real-world application standards.